Predictive Analytics Assignment 6

746 - Subhodeep Bhattacharjee

2026-03-15

Problem Set 6: Model Selection

To demonstrate the application of subset selection methods

Consider the "Hitters" data. For loading the data you need to install ISLR library. We wish to predict a baseball player's salary on the basis of various predictors. So first we need to choose an optimal model.

```
library(ISLR)

## Warning: package 'ISLR' was built under R version 4.2.3

data(Hitters)
```

1. Clean the data by removing all rows that have missing value in any variable.

```
hitters_clean = na.omit(Hitters)

dim(hitters_clean)

## [1] 263  20
```

Here, all the rows with missing values are removed and we now proceed with the cleaned dataset.

2. Split the data randomly into train-test keeping 80% as training sample and remaining as test sample.

```
set.seed(123)

n = nrow(hitters_clean)

train_index = sample(1:n, 0.8*n)

train = hitters_clean[train_index, ]
test  = hitters_clean[-train_index, ]
```

Through random sampling of the cleaned data, 80% is taken as train data that helps us train/build the model while the rest 20% is taken as test data which is used to test the accuracy the model built.

3. Perform the best subset selection on the training sample. While choosing the best model fit up to a 19 variable model. Choose the best model according to adjusted R

squared, BIC , AIC and Cp values. In case the best model differs according to the selection criteria, report the best fitted model according to each selection criterion.

```
library(leaps)

## Warning: package 'leaps' was built under R version 4.2.3

best_model = regsubsets(Salary ~ ., data = train,nvmax = 19)
summary(best_model)

## Subset selection object
## Call: regsubsets.formula(Salary ~ ., data = train, nvmax = 19)
## 19 Variables  (and intercept)
##            Forced in Forced out
## AtBat          FALSE      FALSE
## Hits           FALSE      FALSE
## HmRun          FALSE      FALSE
## Runs           FALSE      FALSE
## RBI            FALSE      FALSE
## Walks          FALSE      FALSE
## Years          FALSE      FALSE
## CAtBat         FALSE      FALSE
## CHits          FALSE      FALSE
## CHmRun         FALSE      FALSE
## CRuns          FALSE      FALSE
## CRBI           FALSE      FALSE
## CWalks         FALSE      FALSE
## LeagueN        FALSE      FALSE
## DivisionW      FALSE      FALSE
## PutOuts        FALSE      FALSE
## Assists        FALSE      FALSE
## Errors         FALSE      FALSE
## NewLeagueN     FALSE      FALSE
## 1 subsets of each size up to 19
## Selection Algorithm: exhaustive
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns
CRBI
## 1  ( 1 )  " "   " "  " "   " "  " " " "   " "   " "    " "   " "    "*"
" "
## 2  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    " "
"*"
## 3  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    " "
" "
## 4  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 5  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 6  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 7  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
```

```
## 8  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 9  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 10  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
"*"
## 11  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
"*"
## 12  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
"*"
## 13  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    " "   " "    "*"
"*"
## 14  ( 1 ) "*"   "*"  " "   "*"  " " "*"   "*"   "*"    " "   " "    "*"
"*"
## 15  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 16  ( 1 ) "*"   "*"  " "   "*"  " " "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 17  ( 1 ) "*"   "*"  "*"   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 18  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 19  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
##           CWalks LeagueN DivisionW PutOuts Assists Errors NewLeagueN
## 1  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 2  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 3  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 4  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 5  ( 1 )  " "    " "     "*"       " "     " "     " "    " "
## 6  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 7  ( 1 )  " "    " "     "*"       "*"     "*"     " "    " "
## 8  ( 1 )  " "    " "     "*"       "*"     "*"     "*"    " "
## 9  ( 1 )  " "    " "     "*"       "*"     "*"     "*"    "*"
## 10  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    " "
## 11  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    "*"
## 12  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 13  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 14  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 15  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 16  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 17  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 18  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 19  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    "*"
```

Here, Salary is taken as the response variable and rest are predictors.
Best Subset Selection is basically checking all possible combinations of predictors and finds the best model according to criteria like:
- Adjusted R^2

- BIC
- AIC
- Mallows Cp

```
best_summary = summary(best_model)
best_summary$adjr2

##  [1] 0.3916432 0.4742947 0.5077294 0.5327750 0.5492949 0.5541373 0.5573872
##  [8] 0.5609618 0.5631858 0.5668744 0.5702673 0.5721978 0.5720259 0.5710908
## [15] 0.5696920 0.5678958 0.5659643 0.5638778 0.5615944

best_summary$bic

##  [1]  -94.68167 -121.01098 -130.48038 -137.12086 -140.36017 -138.31349
##  [7] -135.53972 -132.93765 -129.70443 -127.19075 -124.55313 -121.21484
## [13] -116.85207 -112.12077 -107.16960 -102.03301  -96.84021  -91.58262
## [19]  -86.24126

best_summary$cp

##  [1] 82.632753 44.219902 29.310338 18.476068 11.723250 10.452933  9.938504
##  [8]  9.290045  9.274000  8.603345  8.083007  8.235300  9.336337 10.776077
## [15] 12.416727 14.225938 16.086206 18.005194 20.000000

# regsubsets does not give AIC directly so we use compute manually
rss = best_summary$rss
n = nrow(train)

aic = n*log(rss/n) + 2*(1:19)
```

Now we move on to find the best subset according to different measures, so:

- Adjusted R^2 has to be max for the best model

```
coef(best_model, which.max(best_summary$adjr2))

##  (Intercept)        AtBat         Hits        Walks       CAtBat
CRuns
##  154.1594365   -1.8722264    5.7547106    6.3852096   -0.2092154
1.7712871
##         CRBI       CWalks    DivisionW      PutOuts      Assists
Errors
##    1.1419024   -0.8989348 -121.8633686    0.2013298    0.5363804   -
6.4726568
##   NewLeagueN
##   75.8378982
```

Therefore this model comes out to be best when we take adj R^2.

- BIC, AIC and Cp has to be min for best model

```
# Best model according to  BIC
coef(best_model, which.min(best_summary$bic))

##  (Intercept)        Walks       CAtBat        CHits       CHmRun
DivisionW
##  149.5853608    5.9582020   -0.5241611    2.0687418    1.7652811 -
129.0165012

# Best model according to AIC
coef(best_model, which.min(aic))

##  (Intercept)        AtBat         Hits        Walks       CAtBat
CRuns
##  154.1594365   -1.8722264    5.7547106    6.3852096   -0.2092154
1.7712871
##         CRBI       CWalks    DivisionW      PutOuts      Assists
Errors
##    1.1419024   -0.8989348 -121.8633686    0.2013298    0.5363804   -
6.4726568
##   NewLeagueN
##   75.8378982

# Best model according to Mallow's Cp
coef(best_model, which.min(best_summary$cp))

##  (Intercept)        AtBat         Hits        Walks       CAtBat
CRuns
##  134.1593585   -1.9328699    5.8271814    6.5037245   -0.2063323
1.7820180
##         CRBI       CWalks    DivisionW      PutOuts      Assists
NewLeagueN
##    1.1120229   -0.8879140 -121.2135304    0.1925628    0.3316973
71.1433129
```

4. Find the training MSE and test MSE for each of the best models selected according to different criterion as mentioned in 3 and compare their values.

```
#Train MSE and Test MSE
m.cp=lm(Salary~AtBat + Hits + Walks + CAtBat + CRuns + CRBI + CWalks +
Division + PutOuts + Assists + NewLeague,data=train)
m.bic=lm(Salary~Walks + CAtBat + CHits + CHmRun + Division,data=train)
m.adjr=lm(Salary~AtBat+Hits+Walks+CAtBat+CRuns+CRBI+CWalks+Division+PutOuts+A
ssists+Errors+NewLeague,data=train)
m.aic=lm(Salary~AtBat + Hits +Walks+ CAtBat + CRuns + CRBI + CWalks +
Division + PutOuts + Assists + Errors + NewLeague,data=train)
mse.train.cp=1/210*(sum((train$Salary-predict(m.cp))^2)) #Training MSE for cp
mse.train.bic=1/210*(sum((train$Salary-predict(m.bic))^2)) #Training MSE for
bic
```

```
mse.train.adjr=1/210*(sum((train$Salary-predict(m.adjr))^2)) #Training MSE
for adjR2
mse.train.aic=1/210*(sum((train$Salary-predict(m.aic))^2)) #Training MSE for
aic
mse.train.bic

## [1] 97026.43

mse.train.adjr

## [1] 88935.8

mse.train.cp

## [1] 89790.63

y.hat.test.cp=predict(m.cp, newdata=test)
y.hat.test.bic=predict(m.bic, newdata=test)
y.hat.test.adjr=predict(m.adjr, newdata=test)
y.hat.test.aic=predict(m.aic, newdata=test)
mse.test.cp=mean((test$Salary-y.hat.test.cp)^2)
mse.test.cp

## [1] 125652.3

mse.test.bic=mean((test$Salary-y.hat.test.bic)^2)
mse.test.bic

## [1] 153090.5

mse.test.adjr=mean((test$Salary-y.hat.test.adjr)^2)
mse.test.adjr

## [1] 130066.9

mse.test.aic=mean((test$Salary-y.hat.test.aic)^2)
mse.test.aic

## [1] 130066.9

out.best=data.frame(Measure=c("Cp","BIC","AdjR2","AIC"),Training_MSE=c(mse.tr
ain.cp,mse.train.bic,mse.train.adjr,mse.train.aic),

Test_MSE=c(mse.test.cp,mse.test.bic,mse.test.adjr,mse.test.aic))
out.best

##   Measure Training_MSE Test_MSE
## 1      Cp     89790.63 125652.3
## 2     BIC     97026.43 153090.5
## 3   AdjR2     88935.80 130066.9
## 4     AIC     88935.80 130066.9
```

Interprtation
Lower training MSE → better fit to training data
AdjR^2 and AIC models fit training data best.
BIC gives the simplest model, so it fits training data less closely.

Lower test MSE → better prediction ability
BIC model underfits,Too simple,High test error
AdjR^2 and AIC models slightly overfit,Very low training error,Test error increases
Cp model generalizes best,Lowest test MSE

Therefore, based on prediction accuracy - Cp-selected model is best.

5. Repeat 3-4 using forward stepwise selection method.

```
#Forward Selection
forward.subset=regsubsets(Salary~.,data=train,nvmax=19,method="forward")
summary(forward.subset)
```

```
## Subset selection object
## Call: regsubsets.formula(Salary ~ ., data = train, nvmax = 19, method =
"forward")
## 19 Variables  (and intercept)
##            Forced in Forced out
## AtBat          FALSE      FALSE
## Hits           FALSE      FALSE
## HmRun          FALSE      FALSE
## Runs           FALSE      FALSE
## RBI            FALSE      FALSE
## Walks          FALSE      FALSE
## Years          FALSE      FALSE
## CAtBat         FALSE      FALSE
## CHits          FALSE      FALSE
## CHmRun         FALSE      FALSE
## CRuns          FALSE      FALSE
## CRBI           FALSE      FALSE
## CWalks         FALSE      FALSE
## LeagueN        FALSE      FALSE
## DivisionW      FALSE      FALSE
## PutOuts        FALSE      FALSE
## Assists        FALSE      FALSE
## Errors         FALSE      FALSE
## NewLeagueN     FALSE      FALSE
## 1 subsets of each size up to 19
## Selection Algorithm: forward
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns
CRBI
```

```
## 1  ( 1 )  " "   " "  " "   " "  " " " "   " "   " "    " "   " "    "*"
" "
## 2  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
" "
## 3  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
" "
## 4  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
" "
## 5  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
"*"
## 6  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
"*"
## 7  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 8  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 9  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 10  ( 1 ) " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 11  ( 1 ) " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 12  ( 1 ) " "   " "  " "   " "  " " "*"   "*"   "*"    "*"   " "    "*"
"*"
## 13  ( 1 ) "*"   " "  " "   " "  " " "*"   "*"   "*"    "*"   " "    "*"
"*"
## 14  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   " "    "*"
"*"
## 15  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 16  ( 1 ) "*"   "*"  " "   "*"  " " "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 17  ( 1 ) "*"   "*"  " "   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 18  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 19  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
##           CWalks LeagueN DivisionW PutOuts Assists Errors NewLeagueN
## 1  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 2  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 3  ( 1 )  "*"    " "     " "       " "     " "     " "    " "
## 4  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 5  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 6  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 7  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 8  ( 1 )  "*"    " "     "*"       "*"     " "     " "    " "
## 9  ( 1 )  "*"    " "     "*"       "*"     " "     " "    "*"
## 10  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    "*"
## 11  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
```

```
## 12  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 13  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 14  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 15  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 16  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 17  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 18  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 19  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    "*"
```

```
rssf = forward.subset$rss
nf = nrow(train)
aicf = nf*log(rssf/nf) + 2*(1:length(rssf))
aicf
```

```
##  [1] 2585.818 2482.442 2456.878 2447.780 2441.461 2433.756 2427.875
2424.394
##  [9] 2422.882 2422.195 2422.458 2421.651 2423.156 2424.621 2421.553
2423.095
## [17] 2424.884 2426.807 2428.640 2430.635
```

```
coef(forward.subset, which.min(summary(forward.subset)$bic)) #Best BIC Model
```

```
##  (Intercept)        Walks       CAtBat        CHits        CRuns
CRBI
##  127.8159745    6.4746441   -0.4127407    1.1133040    1.0341530
0.8520825
##       CWalks    DivisionW
##   -0.5432536 -127.3877839
```

```
coef(forward.subset, which.min(summary(forward.subset)$cp)) #Best cp Model
```

```
##  (Intercept)        Walks       CAtBat        CHits        CRuns
CRBI
##  110.1015291    5.2753942   -0.4498891    1.0510348    1.3425171
0.9381944
##       CWalks    DivisionW      PutOuts      Assists       Errors
NewLeagueN
##   -0.5426157 -125.5159872    0.1609710    0.4662319   -7.6994411
74.6774479
```

```
coef(forward.subset, which.max(summary(forward.subset)$adjr2)) #Best Adjusted
R squared Model
```

```
##  (Intercept)        AtBat         Hits        Walks        Years
CAtBat
##  207.8865937   -1.6603271    4.9552022    6.0893338  -12.4996617   -
0.2504738
##        CHits        CRuns         CRBI       CWalks    DivisionW
PutOuts
##    0.3954684    1.4147636    1.0607231   -0.7225505 -123.4055931
0.1923965
```

```
##      Assists       Errors   NewLeagueN
##    0.5131110   -7.0115407   70.8774554
```

```
coef(forward.subset, which.min(aicf)) #Best AIC Model
```

```
##  (Intercept)        AtBat         Hits        Walks        Years
CAtBat
##  208.9601002   -1.6679101    4.9450314    6.1385686  -12.8940432   -
0.2677079
##        CHits       CHmRun        CRuns         CRBI       CWalks
DivisionW
##    0.6867270    1.0337948    1.1343126    0.6223512   -0.6688210 -
123.4082556
##      PutOuts      Assists       Errors   NewLeagueN
##    0.1927942    0.5367763   -7.2527159   70.5194000
```

```
m.cp3=lm(Salary~Walks+CAtBat+CHits+CRuns+CRBI+CWalks+Division+PutOuts+Assists
+Errors+NewLeague,data=train)
m.bic3=lm(Salary~Walks+CAtBat+CHits+CRuns++CRBI+CWalks+Division,data=train)
m.adjr3=lm(Salary~AtBat+Hits+Walks+Years+CAtBat++CHits+CRuns+CRBI+CWalks+Divi
sion+PutOuts+Assists+Errors+NewLeague,data=train)
m.aic3=lm(Salary~AtBat+Hits+Walks+Years+CAtBat+CHits+CHmRun+CRuns+CRBI+CWalks
+Division+PutOuts+Assists+Errors+NewLeague,data=train)

mse.train.cp3=1/210*(sum((train$Salary-predict(m.cp3))^2)) #Training MSE for
cp
mse.train.bic3=1/210*(sum((train$Salary-predict(m.bic3))^2)) #Training MSE
for bic
mse.train.adjr3=1/210*(sum((train$Salary-predict(m.adjr3))^2)) #Training MSE
for adjR2
mse.train.aic3=1/210*(sum((train$Salary-predict(m.aic3))^2)) #Training MSE
for aic

y.hat.test.cp3=predict(m.cp3, newdata=test)
y.hat.test.bic3=predict(m.bic3, newdata=test)
y.hat.test.adjr3=predict(m.adjr3, newdata=test)
y.hat.test.aic3=predict(m.aic3, newdata=test)
mse.test.cp3=mean((test$Salary-y.hat.test.cp3)^2)
mse.test.bic3=mean((test$Salary-y.hat.test.bic3)^2)
mse.test.adjr3=mean((test$Salary-y.hat.test.adjr3)^2)
mse.test.aic3=mean((test$Salary-y.hat.test.aic3)^2)

out.forward=data.frame(Measure=c("Cp","BIC","AdjR2","AIC"),Training_MSE=c(mse
.train.cp3,mse.train.bic3,mse.train.adjr3,mse.train.aic3),

Test_MSE=c(mse.test.cp3,mse.test.bic3,mse.test.adjr3,mse.test.aic3))
out.forward
```

```
##   Measure Training_MSE Test_MSE
## 1      Cp     90888.44 145923.9
```

```
## 2     BIC     95658.81 146540.9
## 3   AdjR2     88287.08 134001.7
## 4     AIC     88094.46 136065.4
```

Interpretation
AIC model fits the training data best.
BIC model is simplest, so it fits the training data less closely.
Adjusted R^2 model generalizes best (smallest gap).
AIC slightly overfits (best training but worse test performance).
BIC underfits slightly because it strongly penalizes model complexity.
The Adjusted R^2–selected model provides the most accurate prediction of Salary among the considered models.

6. Repeat 3-4 using backward stepwise selection method.

```r
#Backward Selection
backward.subset=regsubsets(Salary~.,data=train,nvmax=19,method="backward")
summary(backward.subset)
```

```
## Subset selection object
## Call: regsubsets.formula(Salary ~ ., data = train, nvmax = 19, method =
"backward")
## 19 Variables  (and intercept)
##            Forced in Forced out
## AtBat          FALSE      FALSE
## Hits           FALSE      FALSE
## HmRun          FALSE      FALSE
## Runs           FALSE      FALSE
## RBI            FALSE      FALSE
## Walks          FALSE      FALSE
## Years          FALSE      FALSE
## CAtBat         FALSE      FALSE
## CHits          FALSE      FALSE
## CHmRun         FALSE      FALSE
## CRuns          FALSE      FALSE
## CRBI           FALSE      FALSE
## CWalks         FALSE      FALSE
## LeagueN        FALSE      FALSE
## DivisionW      FALSE      FALSE
## PutOuts        FALSE      FALSE
## Assists        FALSE      FALSE
## Errors         FALSE      FALSE
## NewLeagueN     FALSE      FALSE
## 1 subsets of each size up to 19
## Selection Algorithm: backward
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns
CRBI
## 1  ( 1 )  " "   " "  " "   " "  " " " "   " "   " "    "*"   " "    " "
```

```
" "
## 2  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    "*"   " "    " "
" "
## 3  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    " "
" "
## 4  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 5  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 6  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 7  ( 1 )  " "   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 8  ( 1 )  "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 9  ( 1 )  "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 10  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 11  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    "*"
" "
## 12  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    "*"
" "
## 13  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    "*"
" "
## 14  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   "*"    "*"
" "
## 15  ( 1 ) "*"   "*"  " "   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
" "
## 16  ( 1 ) "*"   "*"  "*"   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
" "
## 17  ( 1 ) "*"   "*"  "*"   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 18  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 19  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
##           CWalks LeagueN DivisionW PutOuts Assists Errors NewLeagueN
## 1  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 2  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 3  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 4  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 5  ( 1 )  " "    " "     "*"       " "     " "     " "    " "
## 6  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 7  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 8  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 9  ( 1 )  " "    " "     "*"       "*"     "*"     " "    " "
## 10  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    " "
## 11  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    " "
## 12  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    "*"
```

```
## 13  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 14  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 15  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 16  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 17  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 18  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 19  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    "*"
```

```
rssb = backward.subset$rss
nb = nrow(train)
aicb = nb*log(rssb/nb) + 2*(1:length(rssb))

coef(backward.subset, which.min(summary(backward.subset)$bic)) #Best BIC
Model
```

```
##  (Intercept)        Walks       CAtBat        CHits       CHmRun
DivisionW
##  149.5853608    5.9582020   -0.5241611    2.0687418    1.7652811 -
129.0165012
```

```
coef(backward.subset, which.min(summary(backward.subset)$cp)) #Best cp Model
```

```
##  (Intercept)        AtBat         Hits        Walks       CAtBat
CHits
##  151.3709120   -1.5393497    4.6939494    6.1305798   -0.3519984
1.1424820
##       CHmRun        CRuns       CWalks    DivisionW      PutOuts
Assists
##    2.3082206    0.9269527   -0.5968973 -120.5643234    0.1975004
0.5618678
##       Errors   NewLeagueN
##   -7.1940771   71.8060203
```

```
coef(backward.subset, which.max(summary(backward.subset)$adjr2)) #Best
Adjusted R squared Model
```

```
##  (Intercept)        AtBat         Hits        Walks       CAtBat
CHits
##  151.3709120   -1.5393497    4.6939494    6.1305798   -0.3519984
1.1424820
##       CHmRun        CRuns       CWalks    DivisionW      PutOuts
Assists
##    2.3082206    0.9269527   -0.5968973 -120.5643234    0.1975004
0.5618678
##       Errors   NewLeagueN
##   -7.1940771   71.8060203
```

```
coef(backward.subset, which.min(aicb)) #Best AIC Model
```

```
##  (Intercept)        AtBat         Hits        Walks        Years
CAtBat
```

```
##  206.9090835   -1.6223190    4.7968602    6.1025734  -13.6947062   -
0.2912962
##        CHits       CHmRun        CRuns       CWalks    DivisionW
PutOuts
##    1.0879126    2.2788982    0.7909399   -0.5722039 -122.4577604
0.1955745
##      Assists       Errors   NewLeagueN
##    0.5409991   -7.4644748   69.3705806
```

```
m.cp2=lm(Salary~AtBat+Hits+Walks+CAtBat+CHits+CHmRun+CRuns+CWalks+Division+Pu
tOuts+Assists+Errors+NewLeague,data=train)
m.bic2=lm(Salary~Walks+CAtBat+CHits+CHmRun+Division,data=train)
m.adjr2=lm(Salary~AtBat+Hits+Walks+CAtBat+CHits+CHmRun+CRuns+CWalks+Division+
PutOuts+Assists+Errors+NewLeague,data=train)
m.aic2=lm(Salary~AtBat+Hits+Walks+Years+CAtBat+CHits+CHmRun+CRuns+CWalks+Divi
sion+PutOuts+Assists+Errors+NewLeague,data=train)

mse.train.cp2=1/210*(sum((train$Salary-predict(m.cp2))^2)) #Training MSE for
cp
mse.train.bic2=1/210*(sum((train$Salary-predict(m.bic2))^2)) #Training MSE
for bic
mse.train.adjr2=1/210*(sum((train$Salary-predict(m.adjr2))^2)) #Training MSE
for adjR2
mse.train.aic2=1/210*(sum((train$Salary-predict(m.aic2))^2)) #Training MSE
for aic

y.hat.test.cp2=predict(m.cp2, newdata=test)
y.hat.test.bic2=predict(m.bic2, newdata=test)
y.hat.test.adjr2=predict(m.adjr2, newdata=test)
y.hat.test.aic2=predict(m.aic2, newdata=test)
mse.test.cp2=mean((test$Salary-y.hat.test.cp2)^2)
mse.test.bic2=mean((test$Salary-y.hat.test.bic2)^2)
mse.test.adjr2=mean((test$Salary-y.hat.test.adjr2)^2)
mse.test.aic2=mean((test$Salary-y.hat.test.aic2)^2)

out.backward=data.frame(Measure=c("Cp","BIC","AdjR2","AIC"),Training_MSE=c(ms
e.train.cp2,mse.train.bic2,mse.train.adjr2,mse.train.aic2),

Test_MSE=c(mse.test.cp2,mse.test.bic2,mse.test.adjr2,mse.test.aic2))
out.backward
```

```
##   Measure Training_MSE Test_MSE
## 1      Cp     88865.02 134736.7
## 2     BIC     97026.43 153090.5
## 3   AdjR2     88865.02 134736.7
## 4     AIC     88423.82 138693.8
```

Interpretation

AIC model fits training data best.

BIC model is simplest, so it fits the training data least closely.

Cp and AdjR^2 models generalize best (smallest gap).
AIC slightly overfits (lowest training error but higher test error).
BIC underfits due to strong penalty on number of predictors.
Cp and Adjusted R^2 select the optimal predictive model for the dataset.

7. Compare the results of the three methods in terms of choice of best model, train MSE amd test MSE.

Best Subset
Evaluates all possible models
Most computationally expensive
Often gives best fit

Forward Selection
Adds predictors gradually
Faster but may miss optimal model

Backward Selection
Starts with full model
Removes unnecessary predictors

Best subset selection evaluates all possible predictor combinations and generally produces the most optimal model according to statistical criteria such as Adjusted R^2, BIC, AIC and Cp. Forward and backward stepwise selection are computationally efficient alternatives that approximate the best subset results. The final comparison of training and test MSE indicates which model provides the best predictive performance.

Problem Set 7: Shrinkage Methods

Comparison of least squares, ridge regression, LASSO and Elastic Net

You would be working with Hitters data set available in the ISLR library. You have to predict the Salary based on all available variables. First make sure that you have deleted all rows with one or more missing values.

```
rm(list=ls())
library(ISLR)
data(Hitters)

# remove missing values
```

```r
hitters = na.omit(Hitters)
dim(hitters)

## [1] 263  20
```

1. Split the data set into a training set and a test set randomly keeping 100 observations in test set and remaining in the training set.

```r
set.seed(123)

n = nrow(hitters)

test_index = sample(1:n, 100)

train = hitters[-test_index, ]
test  = hitters[test_index, ]
```

2. Train the model using least squares and report the train MSE and test MSE.

```r
library(glmnet)

## Warning: package 'glmnet' was built under R version 4.2.3

## Loading required package: Matrix

## Loaded glmnet 4.1-8

#model.matrix() converts categorical variables into dummy variables
#[, -1] removes intercept column
x_train = model.matrix(Salary ~ ., train)[,-1]
y_train = train$Salary

x_test = model.matrix(Salary ~ ., test)[,-1]
y_test = test$Salary

#Ridge Model
lm_model = lm(Salary ~ ., data = train)

train_pred = predict(lm_model, train)
test_pred  = predict(lm_model, test)

train_mse_lm = mean((train$Salary - train_pred)^2)
test_mse_lm  = mean((test$Salary - test_pred)^2)

train_mse_lm

## [1] 105334.2

test_mse_lm

## [1] 87803.91
```

Train MSE → model fit

Test MSE → prediction accuracy

3. Fit ridge regression models on the training set, with grid values of λ. Report the following:

```
lambda_grid = 10^seq(5, -2, length = 100)

ridge_model = glmnet(x_train, y_train,alpha = 0,lambda = lambda_grid)
```

Explanation:
alpha = 0 → Ridge regression
lambda_grid → penalty values

(i) the dimension of the coefficient matrix so obtained.

```
dim(coef(ridge_model))

## [1]  20 100
```

Interpretation:
rows → predictors + intercept
columns → number of lambda values

(ii) The l2 norm of the regression coefficients corresponding to the 50th and 60th choice of λ. Comment on the output.

```
coef50 = coef(ridge_model, s = lambda_grid[50])
coef60 = coef(ridge_model, s = lambda_grid[60])

norm50 = sqrt(sum(coef50[-1]^2))
norm60 = sqrt(sum(coef60[-1]^2))

norm50

## [1] 155.9135

norm60

## [1] 165.5575
```

Larger λ → stronger shrinkage → smaller L2 norm

(iii) Test MSE corresponding to these two choices of λ.

```r
pred50 = predict(ridge_model, s = lambda_grid[50], newx = x_test)
pred60 = predict(ridge_model, s = lambda_grid[60], newx = x_test)

mse50 = mean((y_test - pred50)^2)
mse60 = mean((y_test - pred60)^2)

mse50

## [1] 89317.28

mse60

## [1] 87580.18
```